



Universidade de Brasília

Departamento de Ciência da Computação

Assembly RISC-V



Até agora:

■ RISC-V

- ⇒ Banco de 32 registradores de 32 bits cada
- ⇒ lw Carrega *words* mas endereça *bytes* na memória
- ⇒ Aritmética somente entre registradores ou imediato

■ Instrução

Significado

add x5, x6, x7

$x5 = x6 + x7$

sub x5, x6, x7

$x5 = x6 - x7$

lw x5, imm(x6)

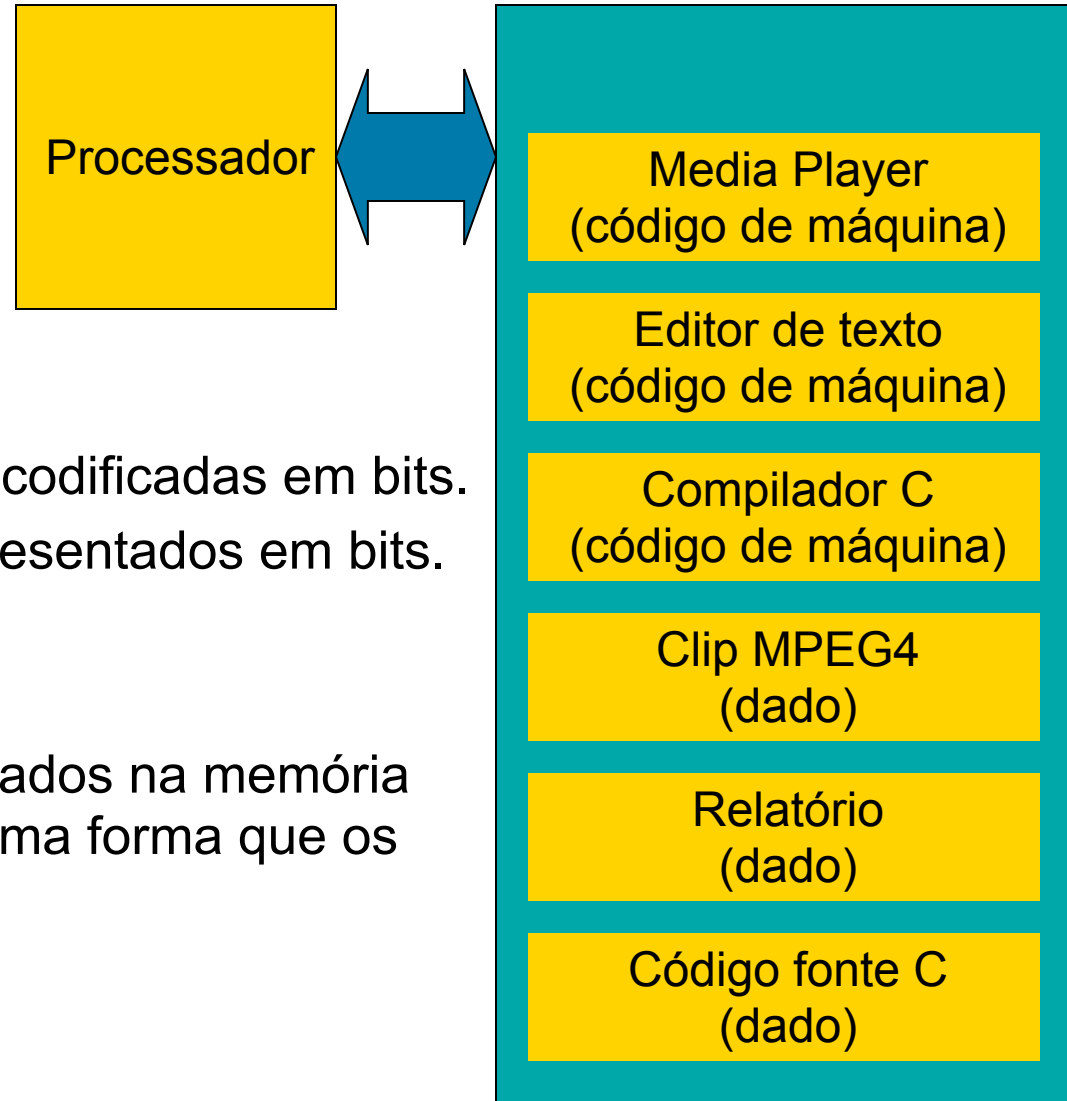
$x5 = \text{Memory}[x6 + \text{imm}]$

sw x5, imm(x6)

$\text{Memory}[x6 + \text{imm}] = x5$



Programa armazenado (conceito)



Todas as instruções são codificadas em bits.
Todos os dados são representados em bits.

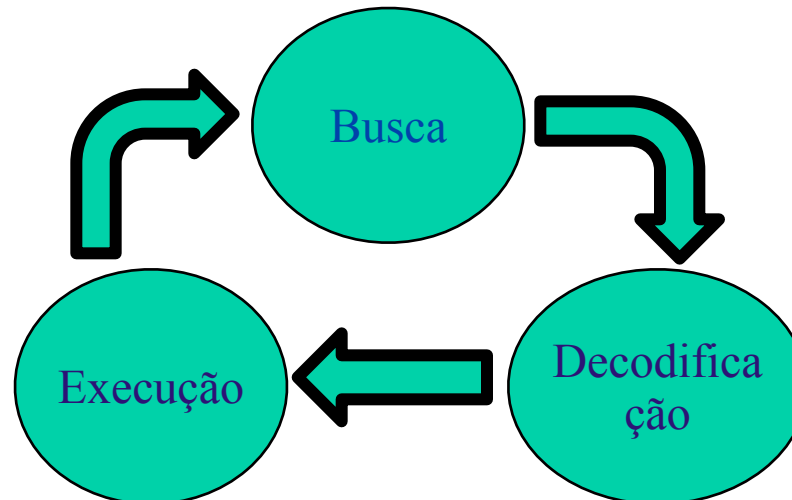
Programas são armazenados na memória
para serem lidos da mesma forma que os
dados.



Programa armazenado (conceito)

Ciclos de busca e execução:

- Instruções são buscadas e colocadas num registrador especial (IR – *Instruction Register*).
- Bits deste registrador "controlam" as ações subseqüentes necessárias à execução da instrução.
- Busca a próxima instrução e continua...





Linguagem de máquina

■ Formato das instruções RISC-V

Name (Field size)	Field						Comments
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

- Tipo R: operações lógico-aritméticas
- Tipo I: operações com dados imediatos pequenos
- Tipo S: operações de armazenamento (*store*)
- Tipo SB: operações de salto condicional
- Tipo U: operações com dados imediatos grandes
- Tipo UJ: operações de salto incondicional



Linguagem de máquina

No RV32I, as instruções, assim como os registradores, também têm 32 bits de comprimento dividido em campos

Formato de instruções com registradores:

funct7	rs2	rs1	funct3	rd	opcode
--------	-----	-----	--------	----	--------

- *opcode 7 bits* código que identifica a instrução
- *rd 5 bits* registrador de operando destino: resultado
- *func3 3 bits* opcode auxiliar de 3 bits
- *rs1 5 bits* primeiro registrador de operando origem
- *rs2 5 bits* segundo registrador de operando origem
- *funct7 7 bits* opcode auxiliar de 7 bits



Linguagem de Máquina

- Exemplo: add t0, s0, s1 # x5 = x8 + x9 (t0, s0, s1)
 - Instrução add: opcode = 0x33
 - funct3 = 0x0
 - funct7 = 0x00
 - Os registradores são identificados por seus índices
- Formato Tipo-R da Instrução

Campo	funct7	rs2	rs1	funct3	rd	opcode
Tamanho	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
hexadecimal	0x00	0x09	0x08	0x0	0x05	0x33
binário	0000 000	0 1001	0100 0	000	0010 1	011 0011



Linguagem de Máquina

■ Instruções com dados imediatos

□ Ex: **addi t0, s0, 15**

■ Formato Tipo-I de instrução:

Campo	Imm[11:0]	rs1	funct3	rd	opcode
Tamanho	12 bits	5 bits	3 bits	5 bits	7 bits
hexadecimal	0x00F	0x08	0x0	0x05	0x13
binário	0000 0000 1111	0100 0	000	0010 1	001 0011

■ Extensão de sinal:

□ Imediato = { 20{imm[11]}, imm[11:0] }

□ Outras instruções Tipo-I:

```

ori t0,s0,0x0F0 # t0= s0 | 0x000000F0 or bit a bit com imediato
lw t0,4(s0) # t0= Mem[s0+4] load word
lbu t0,4(s0) # t0= Mem[s0+4] load byte unsigned
srai t0,s0,2 # t0 = s0 >>> 2 deslocamento aritmético a direita
  
```




Linguagem de Máquina

■ Formato tipo S - Instruções *store*

□ Ex: **sw s0, 16(s1)** # $\text{mem}[\text{s1} + 16] = \text{s0}$

■ Formato Tipo-S de instrução:

Campo	Imm[11:5]	rs2	rs1	funct3	Imm[4:0]	opcode
Tamanho	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
hexadecimal	0x00	0x08	0x09	0x2	0x10	0x23
binário	0000 000	0	0100	010	0100 0	010 0011

■ Extensão de sinal:

- Dado imediato de 12 bits: (20xImm[11], Imm[11:5], Imm[4:0])
- Manter o sinal:
 - 20 x Imm[11] - repete 20 vezes o bit mais significativo
- rs1: registrador de base para calculo do endereço (s1)
- rs2: registrador com dado a ser escrito (s0)



Operações Lógicas

- Operadores lógicos de C e Java e seus correspondentes instruções no RISC-V

Logical operations	C operators	Java operators	RISC-V instructions
Shift left	<<	<<	sll, slli
Shift right	>>	>>>	srl, srli
Shift right arithmetic	>>	>>	sra, srai
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR			or, ori
Bit-by-bit XOR	^	^	xor, xori
Bit-by-bit NOT	~	~	xori



Shift (deslocamento)

0000000	immed	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- Formato I
- immed: quantos bits deslocar
- *shift left logical*: deslocamento lógico à esquerda
 - Desloca os bits à esquerda inserindo zeros à direita
 - Equivale a multiplicar por 2^i
- *shift right logical*: deslocamento lógico à direita
 - Desloca os bits à direita inserindo zeros à esquerda



Operações Lógicas

■ AND

x10	00000000 00000000 00000000 00000000 00000000 00000000 00001101 11000000
x11	00000000 00000000 00000000 00000000 00000000 00000000 00111100 00000000
x9	00000000 00000000 00000000 00000000 00000000 00000000 00001100 00000000

■ OR

x10	00000000 00000000 00000000 00000000 00000000 00000000 00001101 11000000
x11	00000000 00000000 00000000 00000000 00000000 11111111 00000000 11111111
x9	00000000 00000000 00000000 00000000 00000000 11111111 00001101 11111111



Fluxo de Execução de um Programa

- O RISC-V armazena o endereço da instrução a ser executada em um registrador específico
 - *Program Counter*: **PC** - contém o endereço da instrução
- As instruções são armazenadas sequencialmente na memória
 - Instrução atual: $\text{mem}[\text{PC}]$
 - Próxima Instrução: $\text{mem}[\text{PC} + 4]$
 - Busca da Instrução
 - $\text{IR} = \text{mem}[\text{PC}]$ # *Instruction Register*, armazena a instrução
 - $\text{PC} = \text{PC} + 4$ # endereça próxima instrução



Controle de Fluxo de Instrução

- Comando condicionais (*if-then-else*), laços e chamadas de subrotinas são implementados com instruções de desvio
- Desvio Incondicional (*jump*)
 - jal, jalr
- Desvio Condicional (*branch*)
 - beq (se =), bge (se $\geq$), bgeu (se $\geq$ positivos)
 - bne (se $\neq$), blt (se $<$), bltu (se $<$ positivos)



Controle de Fluxo

■ Desvio Incondicional

jal ra, label # Jump and link: ra = PC+4, PC = label

Ex:

```

L1:    add x1, x0, x0
        sub x2, x3, x1
        jal ra, L1                    # jal x0, L1 = jump
  
```

■ Formato Tipo-UJ de instrução :

Campo	Imm[20,10:1,11,19:12]	rd	opcode
Tamanho	20 bits	5 bits	7 bits
hexadecimal	0xFF9FF	0x01	0x6F
binário	1111 1111 1001 1111 1111	0000 1	110 1111

- Salto relativo ao PC: Label = PC + Imm₃₂
- Imm₃₂ = (12x{Imm[20]}, Imm[19:1], 0)



Controle de Fluxo

■ Desvio Incondicional

jalr ra, t0, imm

Jump and link: $ra = PC+4$, $PC = Imm_{32}+t0$

jalr ra, imm(t0)

formato utilizado pelo Patterson

■ Formato Tipo-I de instrução

■ Ex: jalr ra, t0, 4

Campo	Imm[11:0]	rs1	funct3	rd	opcode
Tamanho	12 bits	5 bits	3 bits	5 bits	7 bits
hexadecimal	0x004	0x05	0x0	0x01	0x67
binário	0000 0000 0100	0010 1	000	0000 1	110 0111

■ $Imm_{32} = (20x\{Imm[11]\}, Imm[11:0])$



Controle de Fluxo

Desvio Condicional

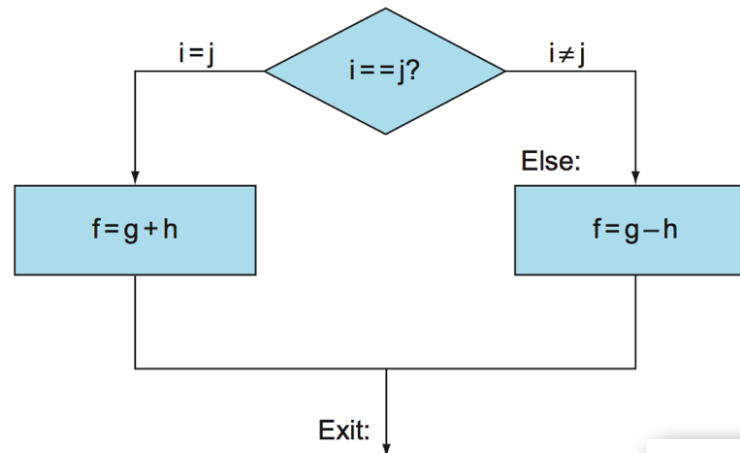
bne t0, t1, *Label* # Branch if Not Equal: $t0 \neq t1$? $PC = Label : PC += 4$
beq t0, t1, *Label* # Branch if Equal: $t0 == t1$? $PC = Label : PC += 4$
bge t0, t1, *Label* # B if greater than/equal: $t0 \geq t1$? $PC = Label : PC += 4$
blt t0, t1, *Label* # B if less than: $t0 < t1$? $PC = Label : PC += 4$

Exemplo:

```

if (i==j)
    f = g+h;
else
    f = g-h;

```



```

bne s4,s5,L1
add s3,s2,s1
jal x0, L2
L1:sub s3,s2,s1
L2:...

```

i: s4, j: s5, f: s3, g: s2, h: s1



Controle de Fluxo

■ Desvio Condicional - Exemplo

□ LABEL: xxxxxxxx
 xxxxxxxx
 xxxxxxxx
 beq t0, t1, LABEL

■ Formato Tipo-SB

Campo	Imm[12,10:5]	rs2	rs1	funct3	Imm[4:1,11]	opcode
Tamanho	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
hexadecimal	0x7F	0x06	0x05	0x0	0x15	0x63
binário	1111 111	0 0110	0010	000	1010 1	110 0011

□ LABEL = PC + { 20x{imm[12]}, imm[11], imm[10:5], imm[4:1], 0}
 = PC + 11111111111111111111 1 111111 1010 0
 = PC - 12



Operadores Relacionais

- `slt t0, t1, t2` # $t0 = 1$ se $t1 < t2$, senão 0 (Tipo-R)
- `slti t0, t1, imm` # $t0 = 1$ se $t1 < \text{imm}$, senão 0 (Tipo-I)
- `sltu t0, t1, t2` # comparação de positivos (Tipo-R)
- `sltiu t0, t1, imm` # compara positivos, (Tipo-I)
 - `sltiu`: `imm` tem o sinal extendido e é tratado como *unsigned*



Constantes

- Constantes são usadas frequentemente

Por exemplo:

$$\begin{aligned}A &= 7283891; \\ B &= A + 1881729383; \\ C &= B / 91827261287854;\end{aligned}$$

- Soluções?
 - colocar “constantes” na memória e carregá-las (lw).
 - criar registradores *hardwired* (como **zero**) para constantes como **um**.
 - colocar as constantes na própria instrução
- Princípio de projeto: agilizar o caso comum.



Uso de Constantes

- Constantes de até 12 bits: Uso das instruções tipo-I

Ex.: `addi t0, t1, 4` # $t0 = t1 + 4$

- Constantes de 13 até 32 bits: Instruções tipo-U

`lui t0, 0x12345` # Load Upper Immediate $t0 = 0x12345000$

`auipc t0, 0x12345` # Add Upper Immediate to PC $t0 = PC + 0x12345000$

- Formato Tipo-U de instrução:**

Campo	Imm[31:12]	rd	opcode
Tamanho	20 bits	5 bits	7 bits
binário	0001 0010 0011 0100 0101	0010 1	011 0111
hexadecimal	0x123452b7		

Imediato = { imm[31:12], 000000000000 }



Pseudo-Instruções

- São instruções implementadas pelo montador, a partir das instruções do processador (como macros). Não estão incluídas na ISA do processador
 - Exemplos:
 - `lw t1, label` `# t1 <= mem[label]`
 - `li t0, 0x123` `# t0 <= 0x00000123`
 - `mv t1, t2` `# t1 <= t2`



Modos de endereçamento

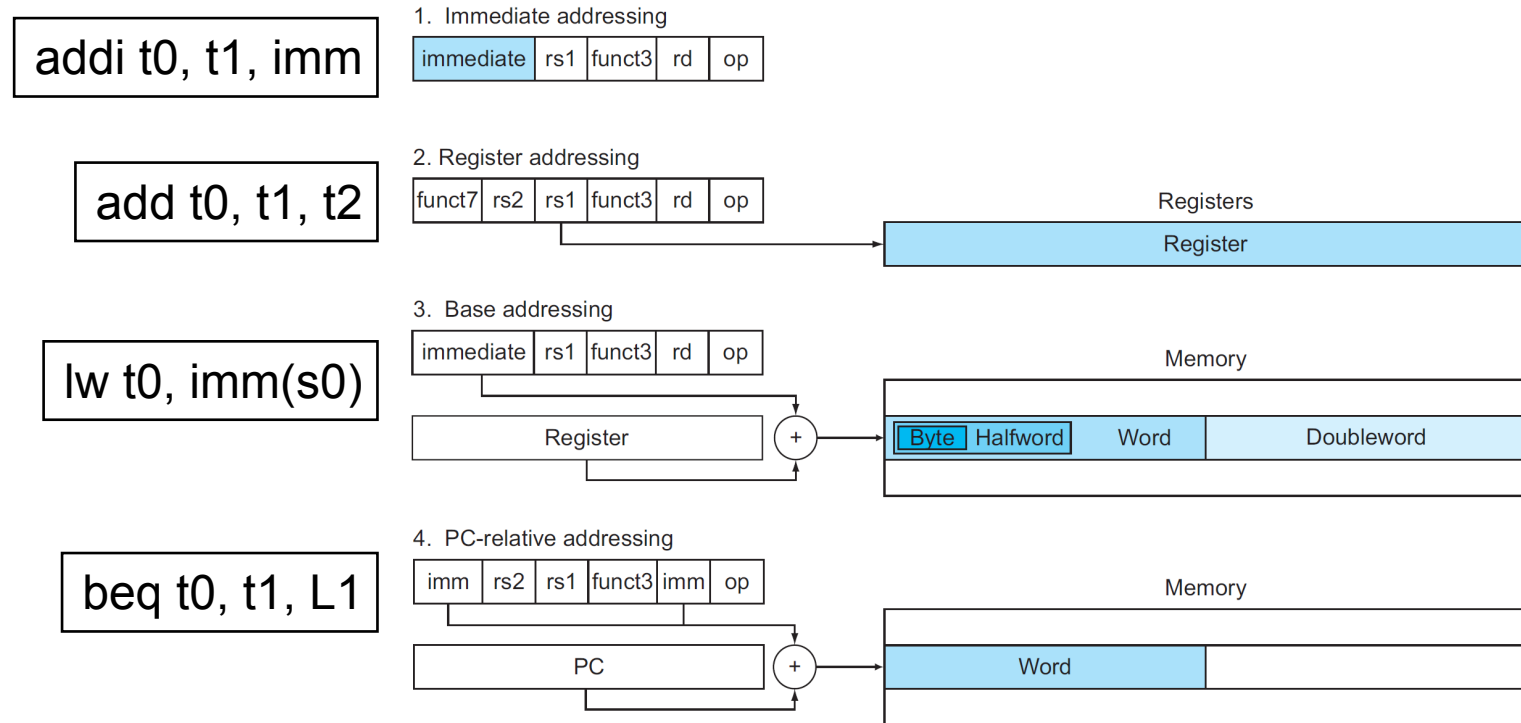


FIGURE 2.17 Illustration of four RISC-V addressing modes. The operands are shaded in color. The operand of mode 3 is in memory, whereas the operand for mode 2 is a register. Note that versions of load and store access bytes, halfwords, words, or doublewords. For mode 1, the operand is part of the instruction itself. Mode 4 addresses instructions in memory, with mode 4 adding a long address to the PC. Note that a single operation can use more than one addressing mode. Add, for example, uses both immediate (`addi`) and register (`add`) addressing.



Endereços em desvios

Linguagem C: while(save[i]==k) i++;

Loop: slli t1,s3,2
 add t1,t1,s6
 lw t0,0(t1)
 bne t0,s5, Exit
 addi s3,s3,1
 j Loop
 Exit: ...

0x00400000	0000000000010		10011	001	00110	0010011
0x00400004	0000000	10110	00110	000	00110	0110011
0x00400008	0000000000000		00110	010	00101	0000011
0x0040000C	0000000	10101	00101	001	01100	1100011
0x00400010	0000000000001		10011	000	10011	0010011
0x00400014	1111111011011111111				00000	1101111
0x00400018	...					

Obs.: j Loop → jal x0,Loop



Imediatos RV

Format	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R-type	func7							rs2					rs1					funct3			rd					opcode						
I-type	11	10	9	8	7	6	5	4	3	2	1	0	rs1					funct3			rd					opcode						
S-type	11	10	9	8	7	6	5	rs2					rs1					funct3			4	3	2	1	0	opcode						
SB-type	12	10	9	8	7	6	5	rs2					rs1					funct3			4	3	2	1	11	opcode						
UJ-type	20	10	9	8	7	6	5	4	3	2	1	11	19	18	17	16	15	14	13	12	rd					opcode						
U-type	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	rd					opcode						

U-type	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	rd				opcode								
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓													
Imm32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	0	0	0	0	0	0	0	0	0	0	0	0	0

UJ-type	20	10	9	8	7	6	5	4	3	2	1	11	19	18	17	16	15	14	13	12	rd										opcode																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					
													↓	↓	↓	↓	↓	↓	↓	↓																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																

SB-type	12	10	9	8	7	6	5	rs2				rs1				funct3			4	3	2	1	11	opcode								
Imm32	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12	11	10	9	8	7	6	5	4	3	2	1	0